Arquitectura de una aplicación generativa, paso a paso (Bloque 8) / Diagrama la arquitectura de tu miniaplicación [Actividad práctica]

✓ **Hecho:** Ver

Diagrama la arquitectura de tu miniaplicación

Ya conoces cada pieza por separado: tus tres funciones (**"armar_peticion"**, **"enviar_peticion"**, **"extraer_texto"**), tu variable de entorno protegida, el manejo de errores que agregaste, y ahora también el ciclo de tokens y el costo de cada petición.

Esta actividad no agrega ninguna pieza nueva: te pide que dibujes, en un solo **diagrama**, cómo esas piezas se conectan entre sí. Es el cierre de este **bloque**, y la base sobre la que vas a comparar modelos y documentar una decisión en el **bloque** siguiente.

Prerrequisitos de la actividad. No necesitas tu entorno de Python para esta actividad: es un ejercicio de diagramación, no de código. Sí necesitas tener a mano, localmente, el repositorio donde ya vive tu módulo (para poder guardar el diagrama ahí y hacer el commit), y una conexión a internet si vas a usar draw.io en el navegador (no hace falta instalar nada).

🛠️ Qué vas a construir en esta actividad

🎯 **El objetivo es un diagrama de arquitectura de tu propia miniaplicación**: un esquema de cajas y flechas que muestre, en orden, el camino completo que recorre una pregunta desde que el usuario la escribe hasta que recibe una respuesta, incluyendo dónde vive tu credencial, dónde se

maneja un error, y dónde se calcula el costo.

No es un diagrama genérico de "cómo funciona una **API**": tiene que ser el diagrama específico de tu módulo, con los nombres reales de tus funciones.

☝ Un diagrama que cualquier otra persona del programa podría presentar sin cambiar nada no cumple el objetivo de esta actividad.


Diagrama de Referencia: Flujo de una Petición
El camino completo desde la pregunta del usuario hasta la respuesta final.
Pregunta del usuario
armar_peticion
arma URL y cuerpo
Lee GEMINI_API_KEY
desde .env
enviar_peticion
envía y recibe respuesta
try / except:
error de conexión
Proveedor (API externa)
genera la respuesta, token a token
extraer_texto
saca el texto de la respuesta
usageMetadata →
calcular_costo
Respuesta final
texto listo para usar

🧩 Los elementos que tu diagrama debe incluir

Independientemente de la herramienta que uses, estos cinco elementos tienen que estar presentes y ser identificables a simple vista:

1 🧩 Tus tres funciones, en el orden en que se llaman entre sí

"armar_peticion", **"enviar_peticion"** y **"extraer_texto"** como cajas separadas y conectadas, no como un único bloque de "tu código".

2 🔑 Dónde vive "GEMINI_API_KEY" y en qué punto exacto se usa

Tu credencial no aparece de la nada: viene de tu archivo **".env"**, y se usa en un punto específico dentro de **"armar_peticion"** o **"enviar_peticion"**, según cómo la organizaste tú.

3 🛡️ Dónde está el manejo de errores

Qué parte exacta del flujo queda envuelta en un **"try"**, y qué tipo de excepción maneja cada **"except"** que ya definiste en tu módulo.

4 💰 Dónde se calcula el costo

En qué punto del flujo entra **"usageMetadata"** y en qué punto se calcula el costo a partir de ese dato, después de que la respuesta ya llegó.

5 🔗 Qué dato exacto cruza cada flecha

No basta con una flecha genérica entre dos cajas: cada flecha debería poder etiquetarse con lo que efectivamente viaja por ahí (la pregunta, la **URL** y el cuerpo de la petición, la respuesta cruda en **JSON**, el texto ya extraído, el costo calculado).

➡️ **Esta lista no es una lista de cajas fijas que tengas que copiar tal cual**: es una lista de preguntas que tu diagrama tiene que responder. Si alguien que no conoce tu código mira el diagrama y puede señalar dónde está cada uno de estos cinco elementos, cumple el objetivo.

Antes de dar por terminado tu diagrama, revisa este checklist:

> Tus tres funciones aparecen como cajas separadas, en el orden en que se llaman.

> Se ve dónde vive "GEMINI_API_KEY" y en qué función exacta se usa.

> Se ve qué parte del flujo está envuelta en un "try", y qué excepción maneja cada "except".

> Se ve en qué punto entra "usageMetadata" y en qué punto se calcula el costo.

> Cada flecha puede etiquetarse con el dato exacto que transporta.

🛠️ Herramienta recomendada: draw.io (diagrams.net)

Para esta actividad, en principio, sirve cualquier herramienta simple de cajas y flechas, e incluso papel y lápiz, fotografiado con buena luz y buena resolución: lo que se evalúa es la claridad y la precisión del contenido, no el acabado visual.

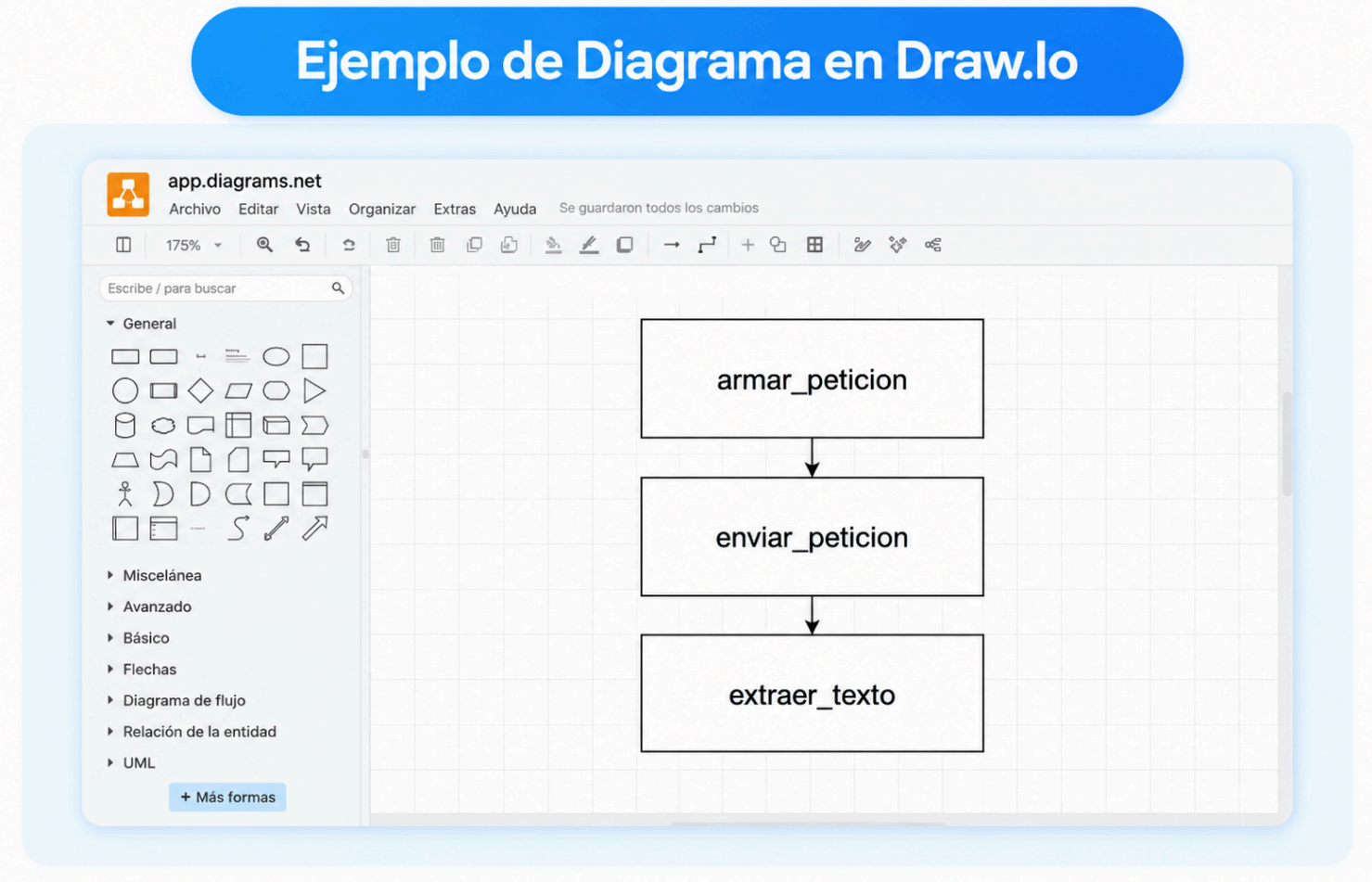
> 💡 Pero si todavía no tienes una herramienta de diagramación propia, la recomendación concreta es **draw.io**, también conocido como **diagrams.net**. Es gratuito, funciona directo en el navegador sin necesidad de crear una cuenta nueva, y puedes abrirlo desde **app.diagrams.net**

Se puede usar sin iniciar sesión, guardando el archivo directo en tu computador, o conectado a tu Google Drive, la misma cuenta que ya usas para tus imágenes en este programa; en cualquiera de los dos casos, exportar el resultado final como PNG para subirlo a tu repositorio es un solo clic.

La razón para recomendarlo específicamente, más allá de esta actividad puntual, es que es una herramienta que te conviene adoptar de una vez para el resto del programa: además de cajas y flechas simples, trae plantillas listas para diagramas de flujo, diagramas de secuencia, e incluso librerías de íconos de arquitecturas cloud (AWS, GCP, Azure) que probablemente vas a necesitar más adelante, cuando la arquitectura de tus proyectos crezca más allá de una miniaplicación.

Aprenderla ahora, con un diagrama simple de tres funciones, evita tener que aprender una herramienta distinta cada vez que el programa te pida un nuevo diagrama.

> ⚠ **Ninguna herramienta específica es obligatoria para aprobar esta actividad**: lo que se evalúa son los cinco elementos de la sección anterior, no el software que uses para dibujarlos. draw.io es, simplemente, la recomendación que te ahorra más tiempo a lo largo de todo el programa, no solo hoy.



📤 Cómo se entrega

Guarda tu **diagrama** como imagen (**PNG** o **JPG**) dentro de tu repositorio, en una carpeta de documentación, y haz un **commit** específico para ese archivo, con un mensaje que lo identifique claramente (por ejemplo "agrega diagrama de arquitectura de la miniaplicación").

```
git add docs/diagrama_arquitectura.png
git commit -m "agrega diagrama de arquitectura de la miniaplicación"
```

```
git push
```

Si tu repositorio todavía no tiene una carpeta para documentación, este es un buen momento para crear una: por ejemplo "docs", al mismo nivel que tu código. Vas a volver a usar esa misma carpeta en la siguiente unidad.

⚠️ Errores frecuentes en esta actividad



Diagramar el flujo genérico de una API, no el tuyo

Un diagrama sin los nombres reales de tus funciones ni la ubicación real de tu credencial no demuestra que entendiste tu propia arquitectura.



Omitir el manejo de errores

Es común diagramar solo el camino feliz (todo funciona) y olvidar dónde entra el **"try/except"** cuando algo falla.



Dejar el costo fuera del diagrama

El costo no es un dato aislado: nace de **"usageMetadata"**, que viaja dentro de la misma respuesta que ya procesa **"extraer_texto"**, y tu diagrama debería mostrar esa relación.



Nota operativa

Si **"git add"** te devuelve algo como **"fatal: pathspec 'docs/tu-archivo.png' did not match any files"**, el problema casi siempre es la ruta: confirma que guardaste la imagen dentro de la carpeta correcta (por ejemplo **"docs"**) en la raíz de tu repositorio, y que el nombre en el comando coincide exactamente con el nombre real del archivo, mayúsculas y extensión incluidas.



Sobre el peso del archivo: no es un problema real para este ejercicio. GitHub recién advierte a partir de **50 MB** y bloquea a partir de **100 MB**, y un PNG de un diagrama de cajas y flechas exportado normalmente pesa, como mucho, unos pocos cientos de KB.

✅ Con este diagrama terminado, cierras el Bloque 8.

Siguiente lección

Bloque 9 · Costos, latencia y escala

En el siguiente bloque vas a comparar dos modelos candidatos por **costo y latencia**, y vas a documentar tu decisión de arquitectura directamente en el repositorio, apoyándote en este mismo diagrama como evidencia de que entiendes el flujo completo, no solo el código línea por línea.

Última modificación: jueves, 24 de septiembre de 2026, 16:17